

ORF307

Linear Optimization

Ioannis Akrotirianakis

Today's topics

- Some simple examples
- Linear optimization (Linear Programming)
- Special cases
- Standard form
- Software and solution methods

Some simple examples

Data fitting example

Dataset: $D = \{(z_i, f_i) \in \mathbb{R}^2 : i = 1, 2, \dots, m\}$

Fit a linear function: $\hat{f}(x) = x_1 + zx_2 = (1, z) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$

Find the best (x_1, x_2) such that

$$f_i \approx \hat{f}(x_i) = x_1 + z_i x_2, \quad \forall i = 1, 2, \dots, m$$

In matrix-vector form:

$$Ax = b \Leftrightarrow \begin{bmatrix} 1 & z_1 \\ 1 & z_2 \\ \vdots & \vdots \\ 1 & z_m \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_m \end{pmatrix}$$

Using L_2 norm:

$$\min \sum_{i=1}^m ((Ax - b)_i)^2 = \|Ax - b\|_2^2$$

Advantage: closed form solution

$$x^* = (A^T A)^{-1} A^T b$$

Disadvantage: sensitive to outliers

The Optimization problem is a QP

QP = Quadratic Programming problem

Data fitting example

Dataset: $D = \{(z_i, f_i) \in \mathbb{R}^2 : i = 1, 2, \dots, m\}$

Fit a linear function: $\hat{f}(x) = x_1 + z x_2 = (1, z) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$

Find the best (x_1, x_2) such that

$$f_i \approx \hat{f}(x_i) = x_1 + z_i x_2, \quad \forall i = 1, 2, \dots, m$$

In matrix-vector form:

$$Ax = b \Leftrightarrow \begin{bmatrix} 1 & z_1 \\ 1 & z_2 \\ \vdots & \vdots \\ 1 & z_m \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_m \end{pmatrix}$$

Using L_1 norm:

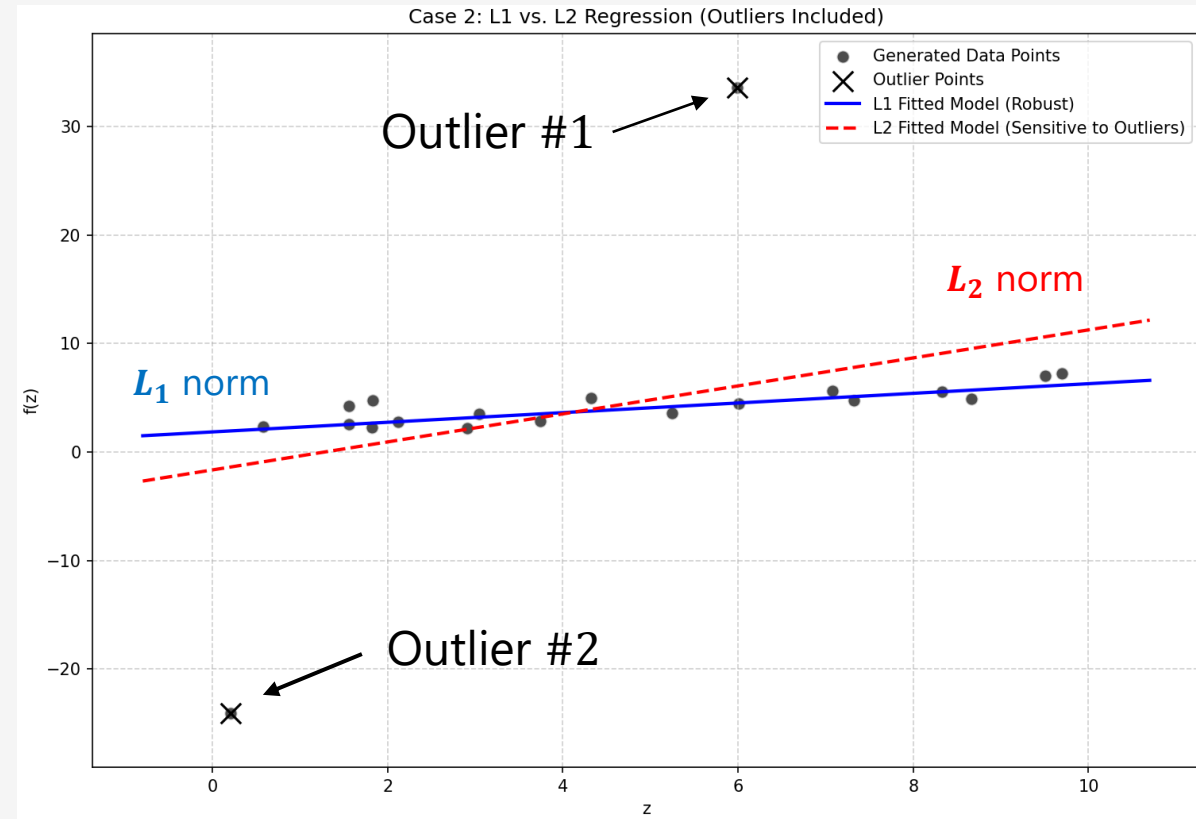
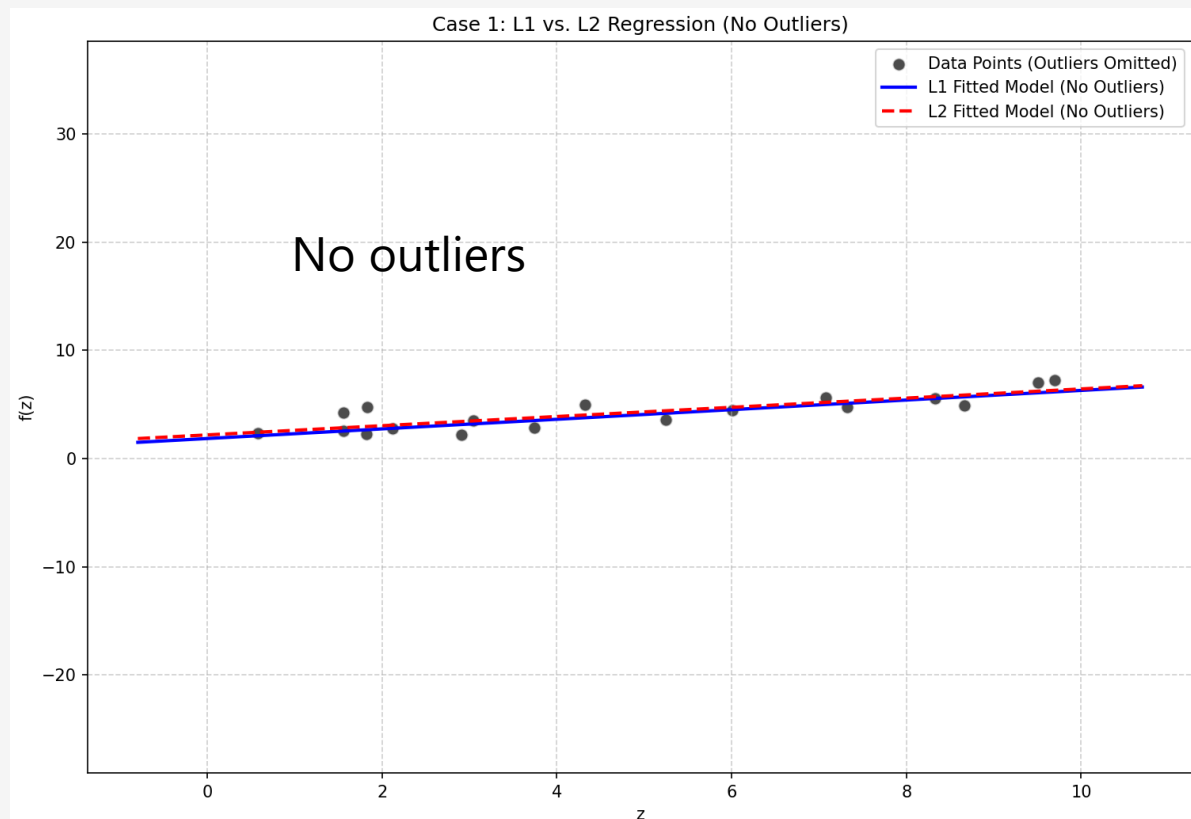
$$\min \sum_{i=1}^m |Ax - b|_i = \|Ax - b\|_1$$

Advantage: x^* is more robust to outliers

Disadvantage: no closed-form solution

The Optimization problem can be converted to an LP

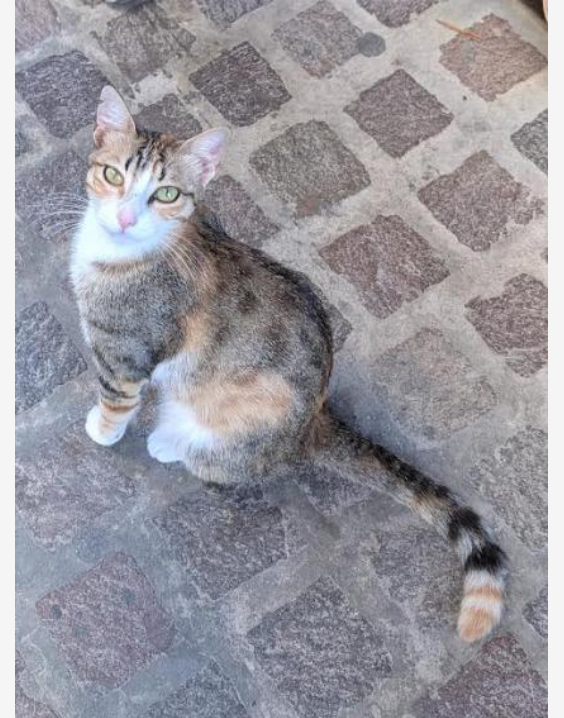
LP = Linear Programming problem



Cheapest cat food problem

- Your cat must have **m** different **nutrients** in its daily diet
- These nutrients exist in **n** **ingredients**
- To prepare her food, you choose **quantities** $x_1, x_2, \dots, x_n$ and mix them
- The **cost** of ingredient x_j is c_j , $j = 1, 2, \dots, n$
- Ingredient j has **nutritional content** a_{ij} of nutrient i
- Your cat requires a **minimum level** of b_i units of nutrient i

$$\begin{aligned} \min_{\mathbf{x}} \quad & \sum_{j=1}^n c_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j \geq b_i, \quad i = 1, \dots, m \\ & x_j \geq 0, \quad j = 1, \dots, n \end{aligned}$$



Linear Optimization (Linear Programming - LP)

Main components and general model

- Main **components** of LP problems:
 - Problem data $\rightarrow c_j, b_i, a_{ij}, d_{ij}, f_i, n, m, p$
 - **Decision variables** $\rightarrow x_1, x_2, \dots, x_n$
 - **Objective function** (linear) $\rightarrow c^T x$
 - **Constraints** (linear) $\rightarrow$
 - ❖ Equalities: $a_i^T x = b_i$
 - ❖ Inequalities: $d_i^T x \leq f_i$

$$\begin{aligned} \min_x \quad & \sum_{j=1}^n c_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j = b_i, \quad i = 1, \dots, m \\ & \sum_{j=1}^n d_{ij} x_j \leq f_i, \quad i = 1, \dots, p \end{aligned}$$

Where can we find Linear Optimization

- Economic planning (its starting point)
- Finance
- Supply chain management & Logistics
- Assignment problems
- Scheduling problems
- Routing problems
- Optimal control problems
- Network design & operations problems
- Military applications (logistics)
- ...

Brief history of Linear Optimization

- **1940s:**
 - Foundations and applications in economics and logistics (Kantorovich & Koopmans)
 - 1947: Simplex method invented by George Dantzig
- **1950s-70s:**
 - Applications expand to Engineering, OR, Computer Science, ...
 - 1975: Nobel prize in economics goes to Kantorovich & Koopmans
- **1980s:**
 - Development of polynomial time algorithms for LP
 - 1979: Ellipsoid algorithm; 1984: Interior Point Method
- **Today:**
 - Continued strong interest in algorithm development of very large problems

Why Linear Optimization

- **Easy to solve**

- LPs can be solved in polynomial time
- Commercial and open-source LP solvers can solve LPs with 10,000s of variables/constraints
- When LPs have special structure we can solve up to 1,000,000s of variables

- **Very versatile methodology**

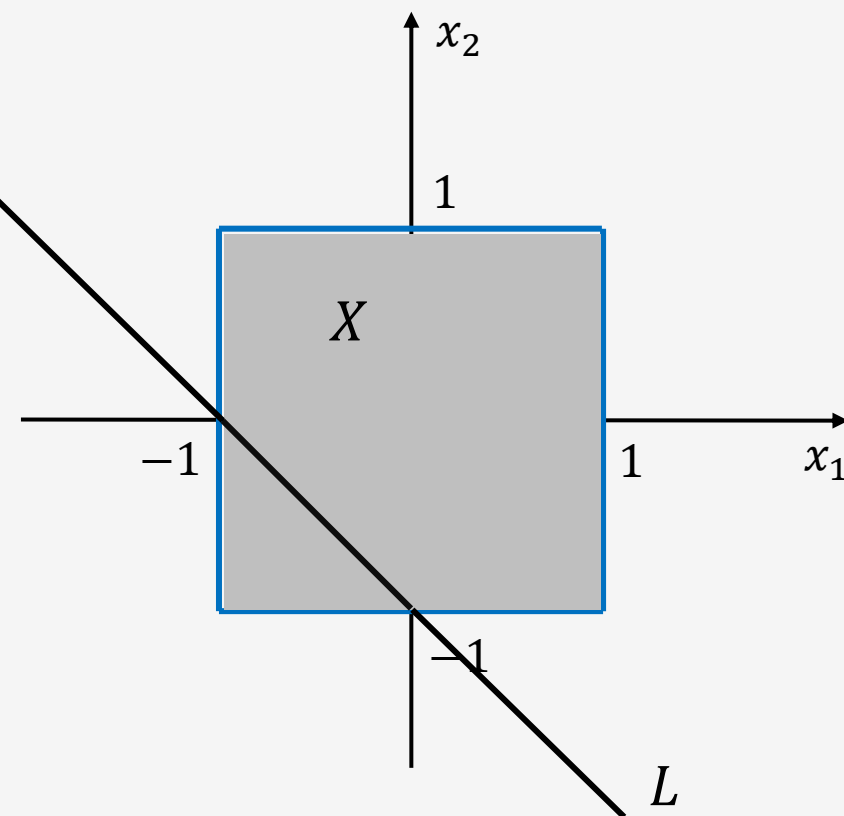
- Many real-world problems can be solved (or approximated) by LP

- **Foundational component of Optimization**

- LP theory lays the foundation for most optimization theories
 - Integer Optimization
 - Nonlinear Optimization

Simple example

Goal: Find a point as far left as possible, in the box X , and restricted in the line L



Simple example

Goal: Find a point as far left as possible, in the box X , and restricted in the line L

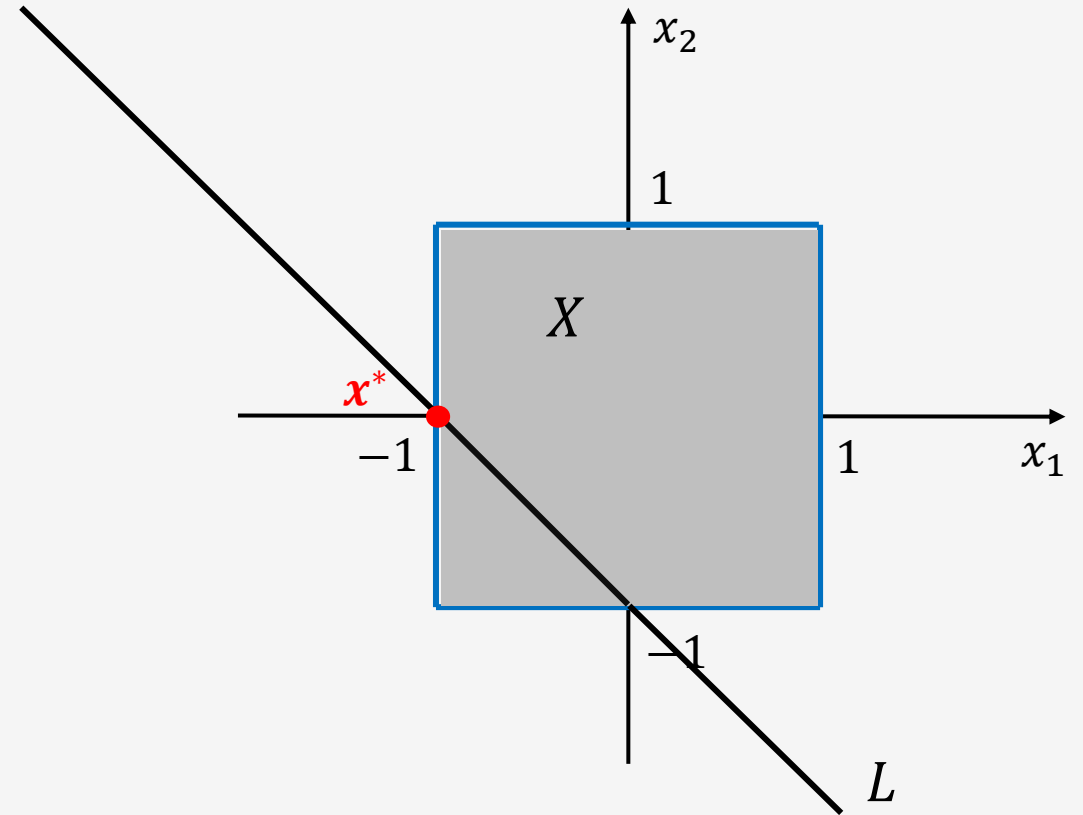
```
import cvxpy as cp

#make decision variable
x = cp.Variable(2)

#define objective
objective = x[0]

#define constraints
constraints = [-1 <= x[0], x[0] <= 1, #inequalities
              -1 <= x[1], x[1] <= 1, #inequalities
              x[0] + x[1] == -1]      #equalities

#make problem and solve
prob = cp.Problem(cp.Minimize(objective), constraints)
prob.solve()
```



Linear Optimization using Vectors

$$\begin{aligned} \min_x \quad & \sum_{j=1}^n c_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j = b_i, \quad i = 1, \dots, m \\ & \sum_{j=1}^n d_{ij} x_j \leq f_i, \quad i = 1, \dots, p \end{aligned}$$



$$\begin{aligned} \min_x \quad & c^T x \\ \text{s.t.} \quad & a_i^T x = b_i, \quad i = 1, \dots, m \\ & d_i^T x \leq f_i, \quad i = 1, \dots, p \end{aligned}$$

$$\begin{aligned} c &= (c_1, c_2, \dots, c_n)^T \\ a_i &= (a_{i1}, a_{i2}, \dots, a_{in})^T \\ d_i &= (d_{i1}, d_{i2}, \dots, d_{in})^T \end{aligned}$$

Linear Optimization using Matrices

$$\begin{aligned} \min_x \quad & \sum_{j=1}^n c_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j = b_i, \quad i = 1, \dots, m \\ & \sum_{j=1}^n d_{ij} x_j \leq f_i, \quad i = 1, \dots, p \end{aligned}$$



$$\begin{aligned} \min_x \quad & c^T x \\ \text{s.t.} \quad & a_i^T x = b_i, \quad i = 1, \dots, m \\ & d_i^T x \leq f_i, \quad i = 1, \dots, p \end{aligned}$$

$$\begin{aligned} c &= (c_1, c_2, \dots, c_n)^T \\ a_i &= (a_{i1}, a_{i2}, \dots, a_{in})^T \\ d_i &= (d_{i1}, d_{i2}, \dots, d_{in})^T \end{aligned}$$



$$\begin{aligned} \min_x \quad & c^T x \\ \text{s.t.} \quad & Ax = b \\ & Dx \leq f \end{aligned}$$

$$\begin{aligned} A &\in R^{m \times n} \\ b &= (b_1, b_2, \dots, b_m)^T \\ D &\in R^{p \times n} \\ f &= (f_1, f_2, \dots, f_p)^T \end{aligned}$$

Optimization Terminology

$$\begin{array}{ll}\min_x & c^T x \\ \text{s.t.} & Ax = b \\ & Dx \leq f\end{array}$$

- **Feasible region:** $\mathcal{F} = \{x \in R^n \mid Ax = b, Dx \leq f\}$
- **Feasible point:** every $x \in \mathcal{F}$ (every x that satisfies the equalities and inequalities in $\mathcal{F}$)
- **Optimal point:** every x^* that satisfies
 - ❖ $x^* \in \mathcal{F}$
 - ❖ $c^T x^* \leq c^T x, \forall x \in \mathcal{F}$
- **Optimal value:** $p^* = c^T x^*$

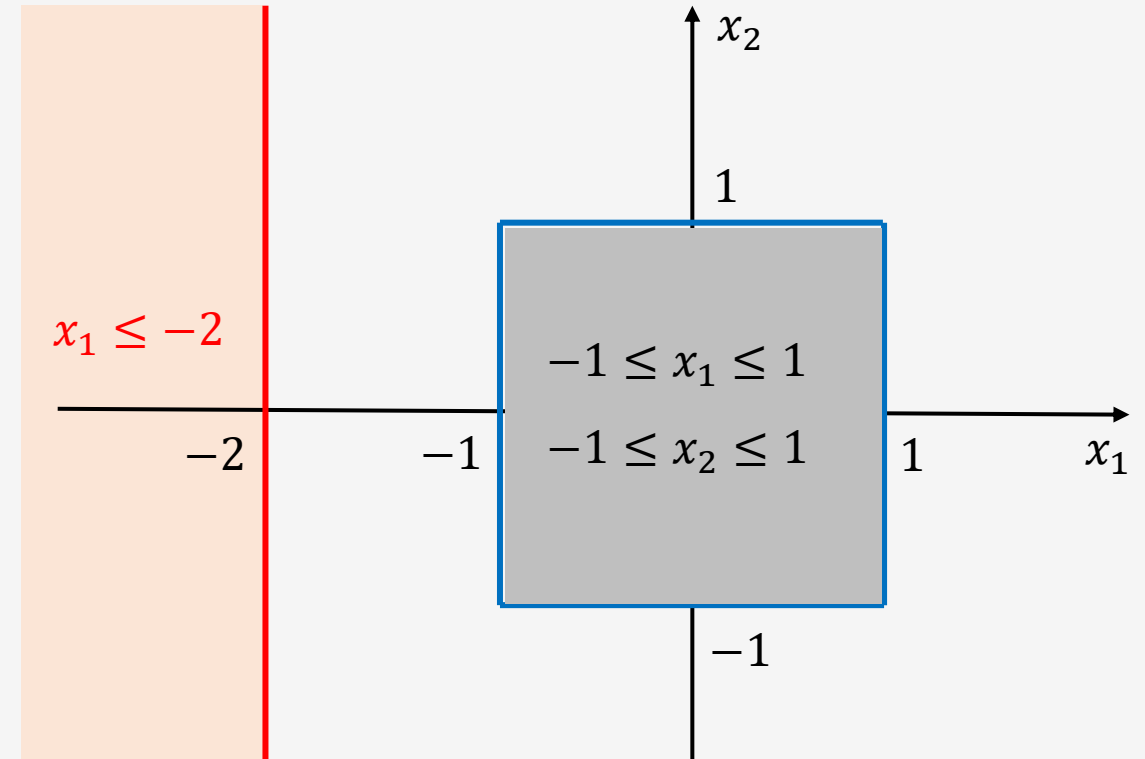
Special cases

Infeasible LP

$$\begin{array}{ll}\min & x_1 \\ \text{s.t.} & -1 \leq x_1 \leq 1 \\ & -1 \leq x_2 \leq 1 \\ & x_1 + x_2 = -1 \\ & x_1 \leq -2\end{array}$$

Remarks

- The feasible region is empty ($\mathcal{F} = \emptyset$)
- The LP is therefore infeasible!
- There is not optimal solution ($\nexists x^*$)
- The optimal value is defined as $p^* = +\infty$

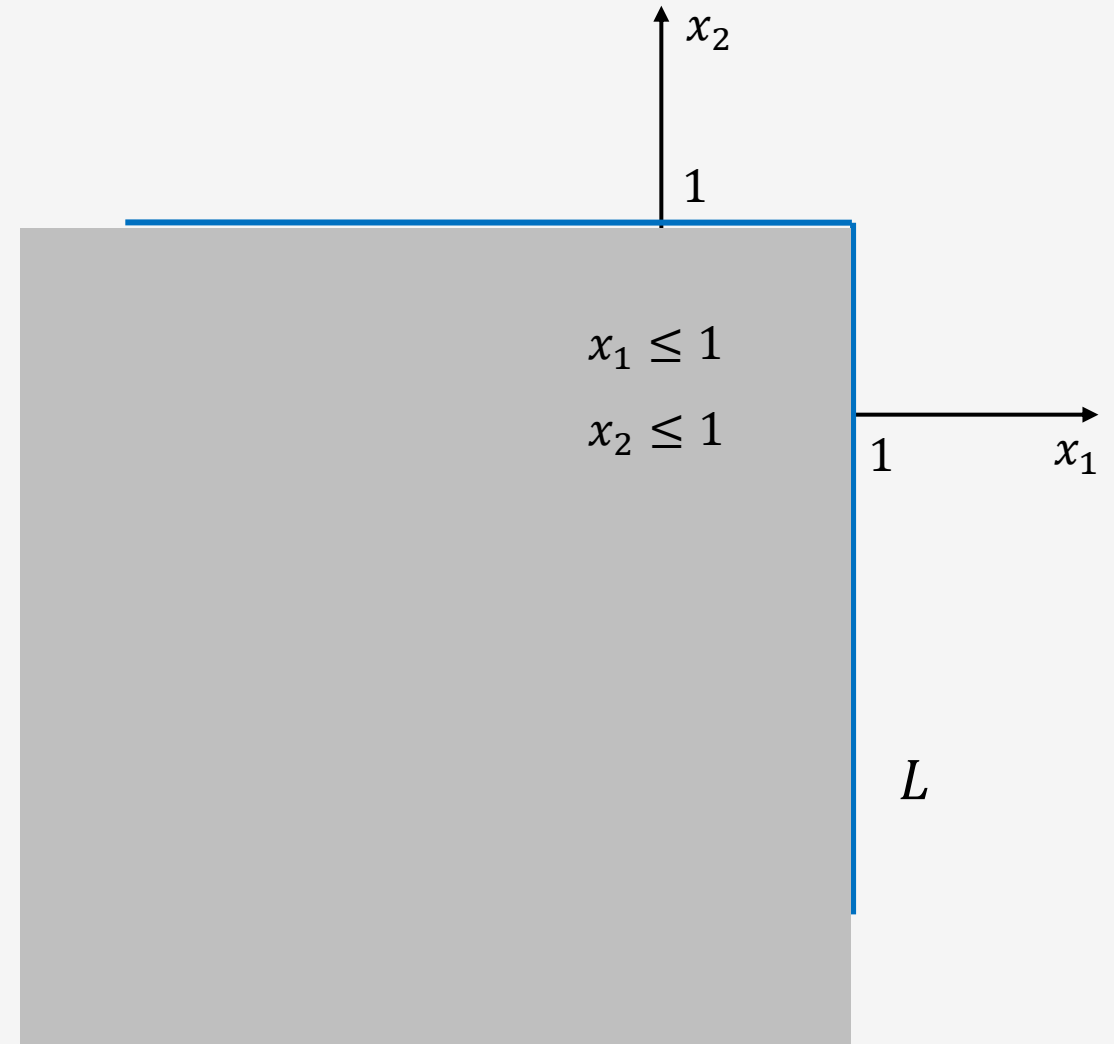


Unbounded LP

$$\begin{array}{ll}\min & x_1 \\ \text{s.t.} & \cancel{-1 \leq x_1 \leq 1} \\ & \cancel{-1 \leq x_2 \leq 1} \\ & \cancel{x_1 + x_2 = -1}\end{array}$$

Remarks

- The feasible region is non-empty ($\mathcal{F} \neq \emptyset$)
- The objective value ($c^T x$) is unbounded below
- The LP is therefore unbounded!
- There is not optimal solution ($\nexists x^*$)
- The optimal value is defined as $p^* = -\infty$



LP with many optimal solutions

$$\min \quad x_1 + x_2$$

$$\text{s.t.} \quad -1 \leq x_1 \leq 1$$

$$-1 \leq x_2 \leq 1$$

$$x_1 + x_2 = -1$$

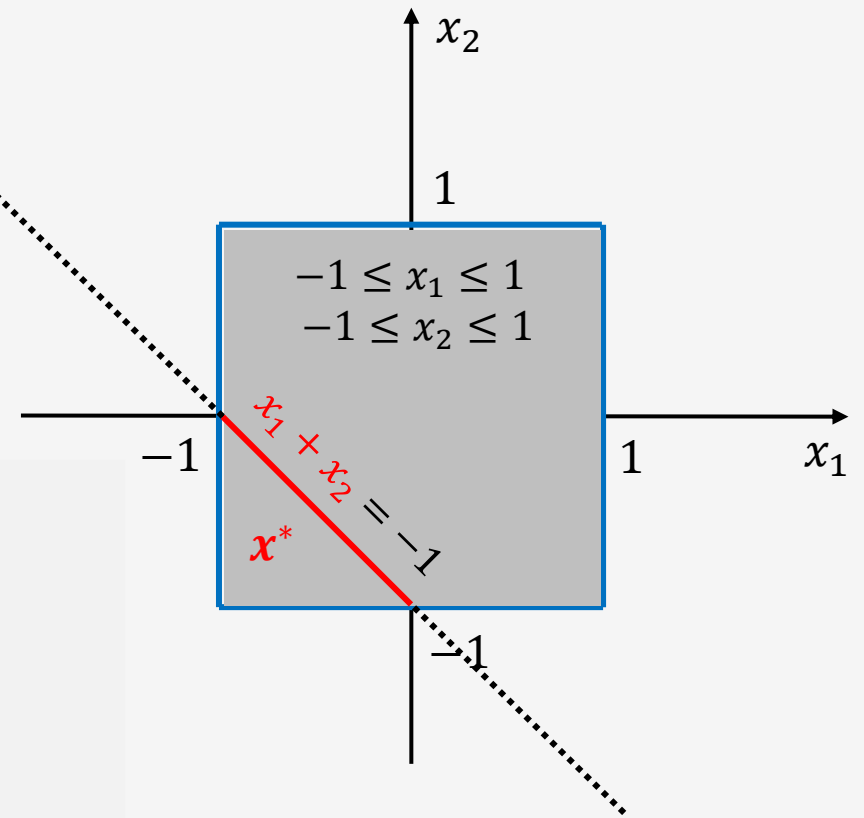
LP with many optimal solutions

$$\min \quad x_1 + x_2$$

$$\text{s.t.} \quad -1 \leq x_1 \leq 1$$

$$-1 \leq x_2 \leq 1$$

$$x_1 + x_2 = -1$$

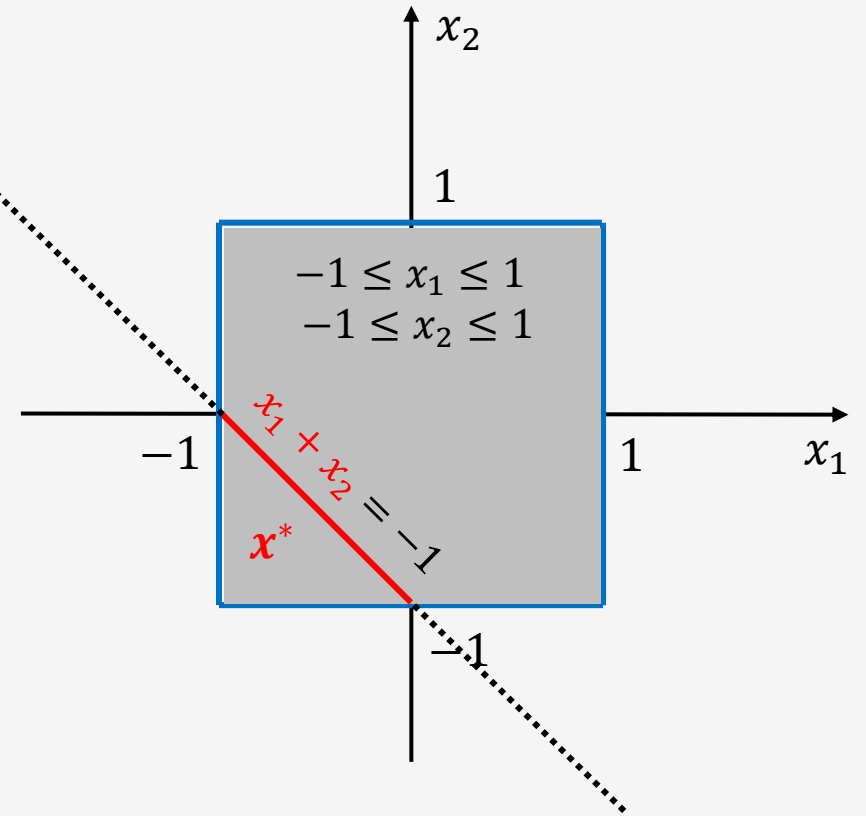


LP with many optimal solutions

$$\begin{array}{ll}\min & x_1 + x_2 \\ \text{s.t.} & -1 \leq x_1 \leq 1 \\ & -1 \leq x_2 \leq 1 \\ & x_1 + x_2 = -1\end{array}$$

Remarks

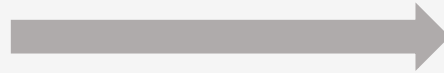
- The feasible region is non-empty ($\mathcal{F} \neq \emptyset$)
- There are infinitely many optimal solutions
 - All points x^* on the line $x_1 + x_2 = -1$
- The optimal value is equal to $p^* = c^T x^* = -1$ for every point on the line $x_1 + x_2 = -1$



Feasibility problems (very important!)

Constrained Satisfiability Problem

$$\begin{array}{ll}\text{find} & x \\ \text{s.t.} & Ax = b \\ & Dx \leq f\end{array}$$



Feasibility Problem

$$\begin{array}{ll}\min_x & 0 \\ \text{s.t.} & Ax = b \\ & Dx \leq f\end{array}$$

Remarks:

- If the constraints are **feasible**, then $p^* = 0$
- Every feasible solution is **optimal**
- If the constraints are **infeasible**, there is no solution, and $p^* = +\infty$

Standard form

LP in Standard Form

Definition:

$$\begin{array}{ll}\min_x & c^T x \\ \text{s.t.} & Ax = b \\ & x \geq 0\end{array}$$

Structure:

- Minimization problem
- Equality constraints
- Nonnegative decision variables

Usage:

- Matrix notation is used in theory
- In algorithm/solver development

LP in Standard Form

Transformation tricks:

- **Change objective**

- ❖ If maximization problem, use $-c^T x$ and convert to minimization

- **Eliminate inequality constraints**

- ❖ If $Ax \leq b$, define slacks variables s and write $Ax + s = b, s \geq 0$

- ❖ If $Ax \geq b$, define surplus variables s and write $Ax - s = b, s \geq 0$

- **Change variable signs**

- ❖ If $x_i \leq 0$, then define $y_i = -x_i$ which is nonnegative

- **Eliminate “free” variables**

- ❖ If x_i is unconstrained (free), define $x_i = x_i^+ - x_i^-$ with $x_i^+, x_i^- \geq 0$

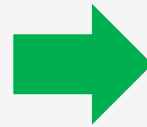
$$\begin{array}{ll} \min_x & c^T x \\ \text{s.t.} & Ax = b \\ & x \geq 0 \end{array}$$

LP in Standard Form

Transformation example:

NOT Standard Form:

$$\begin{array}{ll}\min_{\mathbf{x}} & 2x_1 + 4x_2 \\ \text{s.t.} & x_1 + x_2 \geq 3 \\ & 3x_1 + 2x_2 = 14 \\ & x_1 \geq 0\end{array}$$



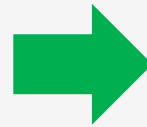
Standard Form:

LP in Standard Form

Transformation example:

NOT Standard Form:

$$\begin{array}{ll}\min_{\mathbf{x}} & 2x_1 + 4x_2 \\ \text{s.t.} & x_1 + x_2 \geq 3 \\ & 3x_1 + 2x_2 = 14 \\ & x_1 \geq 0\end{array}$$



Standard Form:

$$\begin{array}{ll}\min_{\mathbf{x}} & 2x_1 + 4x_2^+ - 4x_2^- \\ \text{s.t.} & x_1 + x_2^+ - x_2^- - x_3 = 3 \\ & 3x_1 + 2x_2^+ - 2x_2^- = 14 \\ & x_1, \quad x_2^+, \quad x_2^-, \quad x_3 \geq 0\end{array}$$

Software

Solvers for LP

Algorithms and theory are very mature

- Simplex method, Interior-Point Methods, First Order Methods, etc.

$$\begin{array}{ll}\min_x & c^T x \\ \text{s.t.} & Ax = b \\ & x \geq 0\end{array}$$

Software is widely available

- Can solve LPs up to several millions of decision variables
- Widely used in industry and academic research

Example solvers

- Commercial solvers: CPLEX, Gurobi, Matlab, Mosek.
- Open-source solvers: GLPK, CLP, SCS, OSQP.

Modeling tools for LP

Modeling tools simplify the formulation of LPs (and other problems)

- Accept optimization problems in common notation
- Recognize problems that can be converted to LPs
- Automatically convert to input format required by specific LP solver

$$\begin{array}{ll}\min_x & c^T x \\ \text{s.t.} & Ax = b \\ & x \geq 0\end{array}$$

Examples:

- AMPL, GAMS (Modeling languages)
- **CVX**, YALMIP (Matlab)
- **CVXPY**, Pyomo (Python)
- JuMP.jl, Cnvex.jl (Julia)

Simple example

Goal: Find a point as far left as possible, in the box X , and restricted in the line L

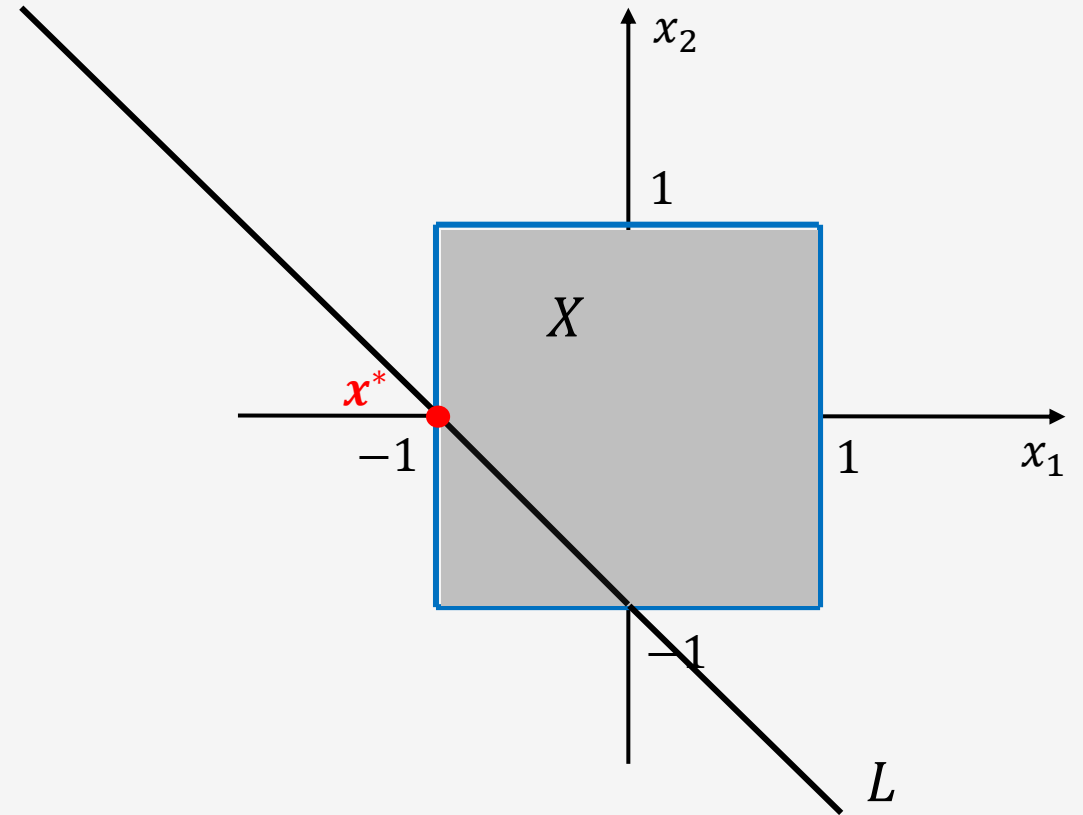
```
import cvxpy as cp

#make decision variable
x = cp.Variable(2)

#define objective
objective = x[0]

#define constraints
constraints = [ cp.norm(x, 'inf') <= 1, #inequalities
               cp.sum(x) == -1]      #equalities

#make problem and solve
prob = cp.Problem(cp.Minimize(objective), constraints)
prob.solve()
```



References

D. Bertsimas & J. Tsitsiklis: Introduction to Linear Optimization
Chapter 1: Introduction

R. Vanderbei: Linear Programming – Foundations & Extensions
Chapter 1: Intro to Linear Programming

Next lecture

- Piecewise Linear Optimization
 - Optimization problems with norms and max functions
 - Some applications
